AP Computer Science Principles

Unit 3 Refresher: Drawing with Code

Today's Objectives:

- Students will be able to describe the CMU canvas and its coordinate system.
- Students will be able to create shapes using arguments and properties.
- Students will be able to explain draw order and use comments.
- Students will be able to print text to the console, including escape sequences.
- Students will be able to tell syntax, runtime, and logic errors apart.

The Canvas

- The CMU canvas is 400 pixels wide and 400 pixels tall.
- The point (0, 0) is the top-left corner.
- x gets bigger as you move to the right.
- y gets bigger as you move down (the opposite of math class).
- The center of the canvas is (200, 200).

Drawing Shapes

- Each shape is made by calling its name with values in parentheses.

```
Rect(50, 50, 100, 80)              # left, top, width, height
Circle(300, 100, 40)               # centerX, centerY, radius
Oval(200, 300, 150, 60)            # centerX, centerY, width, height
Line(0, 400, 400, 0)               # x1, y1, x2, y2
Star(100, 300, 50, 5)              # centerX, centerY, radius, points
RegularPolygon(300, 300, 50, 6)  # centerX, centerY, radius, points
Label('Hi!', 200, 200, size=30)  # value, centerX, centerY
```

Arguments vs. Properties

- Arguments are the required values, in a set order: position and size.
- Properties are optional and written as `name=value`, like `fill='red'`.
- Common properties: `fill`, `border`, `borderWidth`, `opacity`, `rotateAngle`, `dashes`.
- The order of arguments matters. The order of properties does not.
  - `Rect(10, 20, 30, 40)` and `Rect(30, 40, 10, 20)` are different rectangles.

Colors and Gradients

- Colors can be names in quotes, like `'crimson'` or `'skyBlue'`.
- `rgb(red, green, blue)`: each value goes from 0 to 255.
- `gradient(color1, color2)` blends colors. `opacity` goes from 0 (invisible) to 100.

```
Rect(0, 0, 400, 400, fill=gradient('skyBlue', 'white', start='top'))
Circle(200, 200, 50, fill=rgb(255, 200, 0), border='orange', borderWidth=5)
Rect(150, 150, 100, 100, fill='blue', opacity=40)
```

Draw Order

- Python runs code from top to bottom.
- Shapes are drawn in the order they are created, so later shapes appear on top.
- Draw the background first and the details last.
- If a shape "disappears," check whether something drawn later is covering it.

Comments

- # starts a comment. Python ignores the rest of the line.
- Use comments to explain your code and to turn a line off while testing ("commenting out").
- Python's only comment symbol is #. To write a note that spans several lines, put it between triple quotes (""" … """). Python reads it as text but doesn't do anything with it, so it works like a comment.

```
# Draw the sky first so it stays in the back
Rect(0, 0, 400, 400, fill='lightBlue')

# Circle(200, 200, 50)   <- commented out, so it won't run

"""
Sun and grass
drawn below
"""
Circle(350, 50, 30, fill='yellow')
```

Writing to the Console

- `print()` writes text to the console, below the canvas in CMU.
- Text inside quotes is a string. Single or double quotes both work.
- Each `print()` starts a new line. An empty `print()` prints a blank line.

Code

```
print('Hello, world!')
print("It's my first program")
print('Line one')
print()
print('Line three')
```

Console

```
Hello, world!
It's my first program
Line one

Line three
```

Escape Sequences

- A backslash `\` starts an escape sequence: a special character inside a string.

Sequence	Meaning	Example output
`\n`	new line	starts a new line
`\t`	tab	jumps over like the Tab key
`\'`	single quote	' inside a single-quoted string
`\"`	double quote	" inside a double-quoted string
`\\`	backslash	prints one \

Escape Sequences in Action

- Predict the output before running it.

Code

```
print('Go\tTeam!')
print('Line 1\nLine 2')
print("She said \"hi\"")
print('C:\\Users\\csp')
```

Console

```
Go    Team!
Line 1
Line 2
She said "hi"
C:\Users\csp
```

When Things Go Wrong

- Syntax error: the code breaks Python's rules (a missing parenthesis, quote, comma or colon). Nothing runs.
- Runtime error: the code starts running, then hits a problem, like a misspelled name (`NameError`).
- Logic error: the program runs but does the wrong thing, like a shape in the wrong spot.
- Read the error's line number, then check that line and the line just before it.

Check Your Understanding

Think, then share

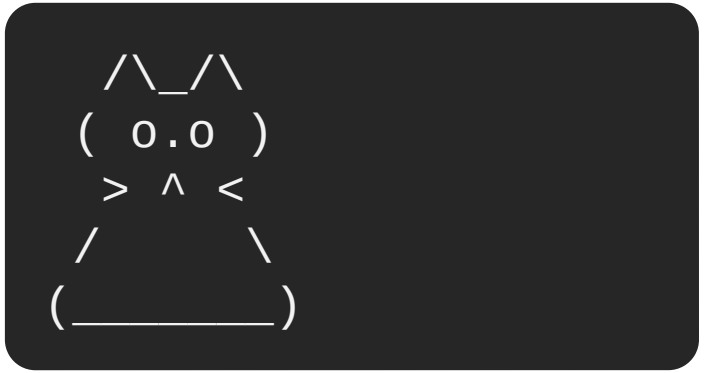
1. Where does `Circle(0, 0, 50)` appear, and how much of it can you see?
2. Two shapes overlap. Which one shows on top?
3. What does `print('A\tB\nC')` display?
4. Find the bug: `Rect(10, 10, 50 50, fill='red')`

Try It: ASCII Art

CMU Sandbox

- Use `print()` statements to draw a picture out of keyboard characters.
- Make it 10 to 20 lines tall.
- Remember: to print one backslash, type `\\`.

Sample output



Adapted from River Hill CompSci, Python – Output exercises

Try It: Initials

CMU Sandbox

- Print your initials as big block letters.
- Build each letter from that letter itself (C's made of C's, B's made of B's).
- Each letter should be 5 lines tall.
- Bonus: draw your initials on the canvas using `Rect` and `Line` shapes.

Sample output

```
CCCC    BBBB
C       B   B
C       BBBB
C       B   B
CCCC    BBBB
```

Adapted from River Hill CompSci, Python – Output exercises

Try It: Poem and Cheer

CMU Sandbox

- Print a poem of at least six lines.
- Use both double quotes and single quotes in the output.
- Indent every other line using `\t`.
- Then print a cheer for a team using at least two escape sequences, and a line with your name, class and teacher separated by backslashes.

Sample output

```
"Roses are red,"
	'Violets are blue,'
"Python is fun,"
	'And so are you!'
"Print one more line,"
	'That makes it six.'
Go	Team!
Jamie\CSP\Ms. Crane
```

Adapted from River Hill CompSci, Python – Output exercises